🧠 1. What "Integration" Really Means

When you integrate Asana, Linear, GitHub, etc. into your app, you're basically:

- **Connecting your app to their API**
- **Authenticating** users so they can let your app access their data
- **Performing actions** (read/write) on their behalf using tokens

So you're not *hosting* Asana or GitHub — you're just making requests to their servers *with permission* from the user.

🔐 2. What OAuth Is (and Why You Need It)

OAuth 2.0 is a system for **secure authorization** — it lets your app act "on behalf" of a user without you ever seeing their password.

Example:

You want your app to:

- Read a user's GitHub repos
- Create issues in Linear
- Sync tasks with Asana

You'll need:

1. **The user to grant permission** (via OAuth)
2. **An access token** (what you'll use to make API calls)
3. Optionally, **a refresh token** (to renew access tokens when they expire)

🔄 3. The Basic OAuth Flow (Standard Across Tools)

Let's say you're connecting to Asana:

1. **Redirect user to Asana's OAuth URL**

```
https://app.asana.com/-/oauth_authorize?client_id=YOUR_CLIENT_ID&
```

2. User logs in and **grants permission**.
3. Asana redirects back to your app with a `code`.
4. You **exchange that code** for an `access_token` (via a POST request to Asana's OAuth endpoint).
5. You store that token in your database (securely).
6. You now make API calls like:

```
GET https://app.asana.com/api/1.0/tasks
Authorization: Bearer ACCESS_TOKEN
```

Same idea for GitHub, Linear, etc. — only the URLs differ.

🧩 4. How to Handle Integrations in General

A. Structure

Each integration usually has:

- **OAuth flow** setup (for login + token exchange)
- **API wrapper** or helper methods to interact with it (e.g., getTasks(), createIssue(), etc.)
- **Webhook listener** (optional — to receive updates from the service)

B. Where to Store Tokens

- Save tokens in your **server database** (never client-side).
- Encrypt them or store in a secure vault (if possible).
- If they expire, use the **refresh token** to get a new one.

C. Sync Logic

Decide what direction your integration goes:

- One-way (you only read)

- Two-way (you read and write back)

Handle conflicts carefully if syncing (e.g., same task edited on both sides).

🧰 5. Practical Example Architecture

Let's say your app is "TaskLinker" and you want Asana + GitHub:

```
Frontend:
  - User clicks "Connect GitHub" → redirected to GitHub OAuth

Backend:
  - /auth/github/callback → handles OAuth callback
  - Exchanges code → gets access_token
  - Saves token in DB for that user

Then:
  - /api/github/repos → fetch repos using stored token
  - /api/github/issues → create new issue, etc.
```

⚙️ 6. Tools You'll Likely Use

Tool	Purpose
Passport.js (Node) or **NextAuth**	Simplify OAuth flow
Axios / Fetch	Make API requests
Redis / DB	Store tokens
Webhooks	Sync live updates back from services
JWT or Session	Manage your own user auth separate from integrations

---

🧭 7. General Strategy for You

1. Start with one integration (e.g., GitHub).
2. Get OAuth working end-to-end.
3. Make one API request to confirm access works.
4. Add database storage for tokens.
5. Build abstraction layers so others (Asana, Linear, etc.) can reuse same flow.
6. Optionally handle refresh tokens + background sync later.